



Development of an AI System for Improving Speaking Skills through Voice Modulation and Tone Practice

**Udvikling af et AI-system til forbedring af taleevner via
stemmemodulation og toneøvelse**

Mikkel Tonatiuh Chavez Petersen
Studienummer: 202109869

15. januar 2026

Aarhus University

Abstract

This bachelor thesis develops an AI based system that helps users improve speaking skills by practicing voice modulation and tone. The main idea is that good speaking is not only about the words, but also about prosody, meaning pitch movement, energy, rhythm, and intonation, the system listens to short speech recordings and classifies them into three speaking styles as: calm, excited, and authoritative. We are group emotions into broader categories that match real coaching goals like calm combines neutral, calm, and sad; excited combines happy, surprised, and fearful; authoritative combines angry and disgusted.

The model is fine tune of a pretrained Wav2Vec2 based speech model using the RAVDESS and CREMA-D datasets. Audio is standardized to a 16 kHz sampling rate and processed through a feature extractor that handles normalization and padding, we improve robustness for real world conditions, training uses data augmentation such as time stretching, pitch shifting, additive noise, volume scaling, mixup, and masking. Class imbalance is fixed using class weights, and we track performance using both accuracy and Unweighted Average Recall (UAR) to see balanced detection across the three styles.

Our results show that the system learns to separate the three styles reasonably well, with calm and authoritative being detected more reliably than excited. The main problem is that excited is often confused with calm or authoritative which overlaps in high energy prosody. in the thesis we discusses how these results translate into practical app feedback, including confidence scoring and prosody tips such as pitch variance.

Abstrakt

Denne bacheloropgave udvikler et AI baseret system som hjælper brugere med at blive bedre til at snakke ved at øve stemmemodulation og tone. Den store ide er at god tale ikke kun handler om ordene men også om prosodi altså hvordan tonehøjde bevægelse, energi, rytme og intonation lyder, systemet lytter til korte tale optagelser og klassificere dem i tre talestile som: rolig, begejstret og autoritativ. Vi grupper følelser ind i bredere kategorier der passer til rigtige coaching mål som rolig samler neutral, rolig og trist; begejstret samler glad, overrasket og frygtsom; autoritativ samler vred og væmmelse.

Modellen er finetuned af en fortrænet Wav2Vec2 baseret tale model ved brug af RAVDESS og CREMA-D datasæt. Lyd bliver standardiseret til 16 kHz sampling rate og bliver kørt gennem en feature extractor der håndterer normalisering og padding, vi gør den mere robust til virkelig verden forhold, træning bruger data augmentation som tidsstræk, pitch shift, additiv støj, volumen skalering, mixup og masking. Klasse ubalance bliver fixed med klasse vægte og vi tracker performance med både accuracy og Unweighted Average Recall (UAR) for at se mere balanceret detektion på de tre stile.

Resultater viser at systemet lærer at skille de tre stile rimelig godt ad, hvor rolig og autoritativ bliver mere pålideligt fundet end begejstret. Det største problem er at begejstret ofte bliver forvekslet med rolig eller autoritativ fordi det overlapper i høj energi prosodi. I opgaven diskutere vi hvordan de her resultater bliver til praktisk app feedback, inklusiv confidence scoring og prosodi tips som pitch varians.

Contents

Contents	3
1 Introduction	6
2 Literature Review	8
3 Methods	10
3.1 From sound to numbers: what an audio file really is	10
3.2 Collecting the dataset: walking folders and reading labels	11
3.3 Building a dataset object and splitting it fairly	12
3.4 Preprocessing: turning raw waves into model-ready inputs	13
3.5 Augmentation 1: time stretching (speed changes without pitch changes)	14
3.6 Augmentation 2: pitch shifting (voice height changes)	15
3.7 Augmentation 3: additive noise (real microphones are messy)	16
3.8 Augmentation 4: volume scaling (energy changes)	16
3.9 Augmentation 5: Mixup (creating in-between voices with soft labels) . .	17
3.10 Augmentation 6: SpecAugmentstyle masking (forcing context use)	18
3.11 Feature extraction: Wav2Vec2FeatureExtractor	18
3.12 The model: why Wav2Vec2 is a good starting point	18
3.13 GAT-style fusion: why we add it and what it does	19
3.14 LoRA fine-tuning: learning with fewer trainable parameters	21
3.15 Loss function: why KL divergence fits mixup and soft labels	22
3.16 Training setup: Trainer, hyperparameter search, scheduler, and early stopping	23
3.17 Turning model outputs into app feedback	24
3.18 Summary: why these methods fit the project goal	25
4 Results	26
4.1 Class distribution per split: what we trained on and why balance matters	27
4.2 Training and validation loss: what “loss” is measuring and why the curve matters	30
4.3 Validation accuracy and UAR: tracking learning in a way that matches the app	33
4.4 Confusion matrix: turning test errors into a clear story	35

4.5	ROC curves: what happens when we change the decision threshold . . .	38
4.6	Precision-recall curves: when the app should trust a prediction	40
4.7	Pitch variance: connecting classification back to prosody feedback	42
4.8	Closing the loop: what these results mean for the app	45
5	Discussion	47
6	Conclusion	49
	Bibliography	50

1 Introduction

Imagine standing in front of a group, ready to share an idea that could change things. Your words are solid, but something feels off. Maybe your voice comes out flat when you want it to spark excitement or too shaky when you aim for calm confidence, I think thats something we can all relate to, speaking well is not just about what you say it is how you say it. That "how" part, the way your voice rises and falls, speeds up or slows down that is what makes people lean in and listen, and getting better at it? that is the goal of the project.

What makes a voice engaging? If we think about sound itself, when you speak, your vocal cords vibrate, creating waves that travel through the air. The waves have patterns some are high pitched like a whistle, others low like a rumble, the rhythm is how fast or slow those words come out, with pauses that build tension or give room to breathe. Then there is the tone, the emotional flavor, when you sound excited with quick ups and downs, or authoritative with steady, firm delivery. Together these things form what we call prosody. This matters because humans pick up on it instantly, a flat tone can make even great ideas seem boring, while the right modulation can turn simple words into something memorable. e.g. in job presentations, or even chats with friends, nailing prosody helps you connect and persuade.

The tricky part is improving it on your own, you might think of recording yourself and listening back, but spotting what is wrong takes skill. A human coach could help, pointing out if your pitch varies enough or if your rhythm feels natural, but even then coaches cost money, and not everyone has access, plus, you want feedback right away, not after a session. That is where we can leverage technology, but it is not easy. Devices need to understand subtle voice changes quickly and reliably, without fancy microphones or controlled lab setups, and if the end experience is meant to feel fun, like a small game where you practice tones, the AI has to be dependable. if the scoring feels random or unfair, it stops being motivating and it stops teaching you

anything useful.

The goal is to build the core of an AI system that listens to your voice and can give instant feedback on speaking styles like calm, excited, or authoritative, the main focus of this thesis is the part that makes an app possible in the first place like training and evaluating a model that can recognize these styles from raw audio. I initially considered an open-source model called SenseVoiceSmall, but I ultimately went with a strong base model from the wav2vec2 family that is pre trained on large amounts of speech data and commonly adapted for emotion related tasks. I picked this kind of model because it starts from raw audio and learns patterns much like how our ears pick up changes in sound, turning waves into meaningful features without needing hand designed rules, I train it on datasets like RAVDESS and CREMA-D, where actors speak with different emotions, I then grouped them into our three styles to keep it tied to prosody patterns you can actually practice.

The intended use case is a website / app where you get a prompt, speak for a short time, and receive a score and guidance on prosody, such as pitch movement or energy, in the time frame of the bachelor project, the emphasis is on building a reliable foundation for that rather than claiming a fully finished, user tested application. The model, the data pipeline, and the evaluation results form a working prototype that can be integrated into a complete app in the future.

The reason for this is that it makes practice more accessible giving feedback within reach without needing special equipment or a human coach. From a data science view, it is also a concrete example of using machine learning to solve a real world task, taking messy, variable speech signals and turning them into useful, interpretable feedback, while keeping the system lightweight enough for fast responses.

First I look at what others have done in speech emotion recognition and related speech tasks, building from basic ideas to why our approach fits, then, the methods section breaks down each step of the pipeline from first principles, explaining why each part exists and how it works, from turning audio into numbers to training the model in a way that generalizes beyond clean recordings. After that i show results and discuss what worked and what still limits the system, and finish with the bigger picture and the next steps, so by the end we will see how this work builds a solid base for real time speaking practice.

2 Literature Review

First off, what is speech emotion recognition, or SER for short? if we break it down, sound is just vibrations in the air, right? When you talk, those vibrations hit a microphone and get turned into a digital signal, basically a long list of numbers showing how the wave goes up and down over time. SER is about teaching a computer to spot feelings in that signal, like happiness from a bubbly tone or anger from sharp bursts. We bother with this because emotions color how we communicate. In our project we are adapting this to detect speaking styles like calm or excited, which are close cousins to emotions. It helps users tweak their voice to match what they intend, making practice more targeted.

People have been working on SER for years, they used to pull out features from audio, like pitch (how high or low the sound is, measured in hertz, which is cycles per second) or energy (how loud it is, like the waves height). They would then feed these into simple classifiers, sort of like decision trees that ask yes or no questions to guess the emotion. But that approach needed experts to pick the right features, and it missed subtle stuff. A big shift came with deep learning, where models learn features on their own from raw data. Surveys show this evolution e.g. one review by El Ayadi and others from 2011 walks through features, classifiers, and databases, showing how the field grew from basic setups to more complex ones [el2011survey]. A newer one from 2023 covers advances like handling noisy real world speech, which is key for our app since users will record in everyday settings [latif2023survey].

Data is the fuel you cannot train without examples, we leverage RAVDESS, which stands for Ryerson Audio-Visual Database of Emotional Speech and Song. It has 24 actors saying the same sentences in eight emotions, like neutral, happy, or angry. They recorded both audio and video, but we just use the sound. Because it is clean, acted but realistic, and covers a range of intensities. The creators tested it with listeners to make sure the emotions come across right [1]. For our project,

we group emotions into calm (neutral, sad, calm), excited (happy, surprised, fearful), and authoritative (angry, disgusted), which makes things simple but we still capture prosody variations, like calm having steady pitch and excited having more ups and downs. Grouping helps because the practice tool is meant to train broad speaking styles, not label every tiny emotional detail.

Another gem is CREMA-D, or Crowd sourced Emotional Multimodal Actors Dataset. This one has 91 actors from diverse backgrounds speaking sentences in six emotions, like happy or sad, what is cool is they got ratings from crowds online to check how well the emotions were perceived, which adds reliability [2]. It is multimodal too, with face and voice, but again, we stick to audio. We use it because it brings variety in accents and ages, making our model more robust for real users, combining that with RAVDESS, we get thousands of clips without collecting our own data, dodging privacy headaches for a proof-of-concept.

The reason I picked wav2vec 2.0 as the base model is because traditional speech models needed text transcripts to learn, but wav2vec 2.0 learns from raw audio alone, like a baby picking up language by listening. It uses self-supervised learning mask parts of the audio and predict them, teaching itself patterns in sound. Then, I fine-tune it for tasks like emotion detection. The paper by Baevski and team from 2020 explains it, they train on huge unlabeled audio datasets, [3]. This makes sense because it handles prosody well since it captures fine details in waves, like rhythm or intonation, without extra steps. We add a custom head with GAT fusion—graph attention, basically a way to focus on important parts of the sequence, like key moments in a speech that reveal tone. This boosts accuracy for our three classes.

But data alone is not enough we need to tweak it to mimic real life, that's why we need to augment we can think of it as remixing songs to create variety. We use time stretching (speeding up or slowing down without changing pitch, like adjusting a rubber band), pitch shifting (raising or lowering tone, by tweaking frequencies), adding noise (random static to simulate backgrounds), and mixup (blending two clips and their labels, creating inbetween emotions). We do this because real speech varies—people talk fast when excited, or in noisy rooms. Augmentations make the model generalize better, preventing it from memorizing just the training data. Studies show this cuts overfitting and increases performance, with limited samples like ours [latif2023survey].

3 Methods

This is the “how we built it” chapter, the goal is simple to say, but a bit tricky to build: we want a model that listens to a short voice clip and decides whether it sounds calm, excited, or authoritative. Then we want to turn that decision into feedback a person can use while practicing.

To make that happen, we take raw audio files and slowly turn them into something a model can learn from. Along the way we make a bunch of choices. Some choices are about fairness (so that one class doesn't dominate). Some choices are about realism (so the model can handle phone microphones and noisy rooms). Some choices are about efficiency (so we can train and run the model without needing a huge machine).

3.1 From sound to numbers: what an audio file really is

A `.wav` file is a list of numbers. Each number is the air pressure of the sound wave at one tiny moment in time. If the number is big, the wave is tall at that moment, and we usually hear that as louder. If the number is small, the wave is closer to silence.

The audio is sampled at a fixed rate, we use 16,000 samples per second, which is often written as 16 kHz, which means in one second, we store 16,000 measurements of the wave.

A helpful way to think about sampling is that time is continuous in the real world, but a computer needs discrete snapshots. If the sampling rate is f_s , then one sample

is taken every $\frac{1}{f_s}$ seconds. So sample index i represents time

$$\underbrace{t_i}_{\text{time of sample } i} = \underbrace{\frac{i}{f_s}}_{\text{index divided by samples per second}}$$

and the waveform value at that time is

$$\underbrace{w[i]}_{\text{wave amplitude at sample } i}.$$

Why 16 kHz? A basic rule from signal processing says that if you sample at f_s , the highest frequency you can represent is about $\frac{f_s}{2}$. This is often called the Nyquist idea. With $f_s = 16000$, that limit is

$$\underbrace{\frac{f_s}{2}}_{\text{max representable frequency}} = \underbrace{\frac{16000}{2}}_{\text{half of 16000}} = \underbrace{8000}_{\text{Hz}}$$

which covers the most important frequency range for speech and also holding file sizes and compute low.

3.2 Collecting the dataset: walking folders and reading labels

Why we parse filenames

RAVDESS and CREMA-D store emotion information inside the filename, which is convenient because we do not need a separate label file, but it also means we must be careful and consistent when parsing.

We use `os.walk` because these datasets are organized as folders with many files inside, sometimes nested. `os.walk` is a generator that visits every folder, then every file in that folder, which is exactly what we want, we want to find every `.wav` path once, store it, and move on.

As we walk, we do two things for each audio file: we store the full file path, and we store the label we derived from the filename. At the end we have two lists with the same length: one list of paths and one list of labels. Those lists become the dataset.

RAVDESS label parsing and our three style mapping

A RAVDESS filename looks like: 03-01-03-01-01-01-01.wav. Inside that, the emotion code is one of the numbers e.g. 03 often indicates happy in the RAVDESS naming. The exact parsing is simple, split by -, pick the right position, convert to an integer, then map that emotion ID into our grouped label.

The grouping is the key design choice, because instead of trying to predict many separate emotions, we predict three speaking styles that are easier to use in a e.g. coaching app: calm, excited, and authoritative.

The mapping used in this project is:

Calm (class 0): neutral, calm, sad. Excited (class 1): happy, surprised, fearful. Authoritative (class 2): angry, disgusted.

The reason for grouping is not to make the task “easier” in a lazy way, the reason is that our app is not trying to be a psychologist, it is trying to help someone practice how they sound. Many fine-grained emotion labels differ in meaning, but can share similar prosody e.g. happy and surprised are different feelings, but they can both have lively pitch movement, faster rhythm, and high energy. For a speaking-practice, that common pattern is what we want to detect.

So the labels are chosen to line up with prosody knobs a person can actually turn: more pitch movement, more energy, more steady firmness, and so on.

CREMA-D label parsing

CREMA-D uses letter codes inside filenames, like _HAP_ for happy. The logic is the same we read the filename string, detect which code is there and map that to one of the same three style groups, then store the final class ID.

At the end of this we have a big set of labeled audio paths from both datasets all following one unified label scheme.

3.3 Building a dataset object and splitting it fairly

Why Hugging Face Datasets

We use a Hugging Face Dataset because it gives us a clean way to store examples and apply transforms it also works smoothly with the Hugging Face training tools later,

the dataset is basically a table with columns like: `path` and `label`. Each row is one audio clip.

Train/validation/test split with stratification

We split the dataset into training, validation, and test: about 80%, 10%, 10%.

Stratification means we preserve class proportions across splits. The idea is that if excited is about one third of the data overall, then excited should also be about one third of training, one third of validation, and one third of test. that matters because otherwise validation and test might accidentally become easier or harder than training, and then the results would be misleading.

Class imbalance and why we use class weights

Even with stratification, classes can still be a bit imbalanced. If one class has more examples, then during training that class contributes more often to the loss, that means over many updates, that can tilt the model toward predicting that class more often.

the way to reduce that tilt, is to use class weights, the weight formula we use is:

$$\underbrace{w_c}_{\text{weight for class } c} = \frac{\overbrace{\text{total training samples}}^n}{\underbrace{k}_{\text{number of classes}} \cdot \underbrace{n_c}_{\text{samples in class } c}}$$

we can interpret this as: If n_c is small (a rarer class), then w_c becomes larger. If n_c is large (a more common class), then w_c becomes smaller. So mistakes on rare classes count more, and mistakes on common classes count less.

This links directly to the coaching goal, if the model misses excitement often, the app will keep telling an excited speaker that they sound calm or authoritative. That is annoying and makes practice less useful. Weights help push training toward more balanced detection.

3.4 Preprocessing: turning raw waves into model-ready inputs

Loading audio and standardizing sample rate

We load audio with `librosa`. The output is a 1D array of floats:

$$\underbrace{w}_{\text{waveform}} = \left[\underbrace{w[0]}_{\text{sample 0}}, \underbrace{w[1]}_{\text{sample 1}}, \dots \right]$$

If an audio file is not already at 16 kHz, we resample it. Resampling means we change how many samples represent one second while at the same time keeping the sound as similar as possible, we do this so the model always sees the same time scale and frequency scale.

Why we augment data

In a perfect lab dataset, every clip is clean and recorded the same way. In a real app, people record in bedrooms, kitchens, or outside, with different microphones, different distances, and different speaking speeds. If we train only on clean data, the model can learn fragile shortcuts that break in real use.

Augmentations are a way to create realistic variation without collecting new data. We take a real waveform and apply controlled changes that might happen in real life. The label stays the same, because the speaking style should still be the same after small changes in speed, pitch, or noise.

Below are the augmentations and what they teach.

3.5 Augmentation 1: time stretching (speed changes without pitch changes)

Time stretching changes how fast the audio plays without shifting pitch. A simple way to think about it is a time re-mapping.

If we let the original time index be t . If we pick a stretch rate r , then the new time can be written as:

$$\underbrace{t'}_{\text{new time}} = \underbrace{\frac{t}{r}}_{\text{time scaled by } r}$$

If $r > 1$, speech becomes faster. If $r < 1$, speech becomes slower.

The wave must be interpolated because t' will not land exactly on integer sample indices. Interpolation just means we estimate in-between values instead of skipping or repeating samples.

The reason we do this is because rhythm is part of speaking style, but small speed differences should not flip the class. We want the model to detect excited speech even if someone speaks a bit faster than the average excited clip in the dataset.

3.6 Augmentation 2: pitch shifting (voice height changes)

Pitch shifting changes the perceived “height” of the voice, this is useful because different speakers have different pitch ranges and even the same speaker can shift pitch depending on mood, health, or context.

from a first principles perspective we can see it like this: a waveform can be described as a mix of frequencies. The Fourier idea is a way to rewrite a signal in terms of frequency content.

Speech changes over time, so we usually use a short time Fourier transform (STFT) which analyzes small chunks of the signal.

We first take a window function g that selects a small region around time t and then we measure how much of the frequency f exists in that region:

$$\underbrace{W(f, t)}_{\text{time-frequency representation}} = \underbrace{\int_{-\infty}^{\infty}}_{\text{sum over all time}} \underbrace{w(\tau)}_{\text{signal}} \underbrace{g(\tau - t)}_{\text{window centered at } t} \underbrace{e^{-i2\pi f\tau}}_{\text{frequency probe}} d\tau$$

we do not need to implement this by hand because libraries do it but the meaning matters here. This transform gives us a grid, time on one axis, frequency on the other, and values showing how strong each frequency is at each time.

Pitch shifting then means moving frequency upward or downward, if we shift by n semitones, we multiply frequencies by:

$$\underbrace{s}_{\text{frequency scale}} = \underbrace{2^{\frac{n}{12}}}_{12 \text{ semitones per octave}}$$

So a frequency f becomes:

$$\underbrace{f'}_{\text{shifted frequency}} = \underbrace{s}_{2^{n/12}} \cdot \underbrace{f}_{\text{original frequency}}$$

We use small shifts, like $n \in [-2, 2]$, to stay realistic.

The reason is because speaking style is not tied to one absolute pitch. A calm speaker can have high pitch if they are naturally high pitched. An excited speaker can have lower pitch if they are naturally low-pitched. Pitch shifting teaches the model to focus on patterns like pitch movement and energy shape, not just absolute pitch height.

3.7 Augmentation 3: additive noise (real microphones are messy)

noise injection means adding a small random signal to the waveform as:

$$\underbrace{w'}_{\text{noisy waveform}} = \underbrace{w}_{\text{original waveform}} + \underbrace{\epsilon}_{\text{noise}}$$

A simple noise choice is Gaussian noise which means ϵ is drawn from a normal distribution with mean 0 and some small standard deviation σ :

$$\underbrace{\epsilon}_{\text{noise}} \sim \underbrace{\mathcal{N}(0, \sigma^2)}_{\text{normal distribution}}$$

we do this because the app will hear air conditioners, keyboard clicks, distance effects, and phone mic hiss. Noise augmentation teaches the model not to panic when the signal is imperfect.

3.8 Augmentation 4: volume scaling (energy changes)

Volume scaling is even simpler:

$$\underbrace{w'}_{\text{scaled waveform}} = \underbrace{v}_{\text{volume factor}} \cdot \underbrace{w}_{\text{original waveform}}$$

where v is sampled from a range like $[0.8, 1.2]$.

we do this because loudness depends on distance to the microphone and recording setup, not only on emotion, we want the model to use energy patterns in context, not treat volume alone as the label.

3.9 Augmentation 5: Mixup (creating in-between voices with soft labels)

Mixup is one of the most important augmentations here because it changes both the input and the label, the intuition is this: real speech styles are not always pure. A person can sound calm but with a bit of excitement. Or authoritative but also friendly. with mixup we create training examples in the middle, which helps the model learn smoother boundaries.

lets pick two training examples (w_1, y_1) and (w_2, y_2) . Then we sample a mixing coefficient λ between 0 and 1:

$$\underbrace{\lambda}_{\text{mix coefficient}} \sim \underbrace{\text{Beta}(\alpha, \alpha)}_{\text{beta distribution}}$$

and we used $\alpha = 0.2$.

The beta distribution is useful because it can prefer values near 0 and 1 when $\alpha < 1$, which means many mixed examples are “mostly one sample plus a little bit of the other”, which sounds more realistic than always mixing 50/50.

The mixed waveform are:

$$\underbrace{w_{\text{mix}}}_{\text{mixed waveform}} = \underbrace{\lambda}_{\text{keep some of } w_1} \cdot \underbrace{w_1}_{\text{sample 1}} + \underbrace{(1 - \lambda)}_{\text{keep some of } w_2} \cdot \underbrace{w_2}_{\text{sample 2}}$$

And the mixed label is also blended:

$$\underbrace{y_{\text{mix}}}_{\text{soft label}} = \underbrace{\lambda}_{\text{label weight 1}} \cdot \underbrace{y_1}_{\text{one-hot or soft label}} + \underbrace{(1 - \lambda)}_{\text{label weight 2}} \cdot \underbrace{y_2}_{\text{onehot or soft label}}$$

If the two waveforms have different lengths, we pad the shorter one with zeros so the addition is defined, zerro padding means adding silence at the end.

mixed matters for this project because the app will often hear borderline tones. Mixup gives the model practice on borderline cases in training, instead of forcing every training example to be a perfect.

3.10 Augmentation 6: SpecAugmentstyle masking (forcing context use)

After feature extraction, we can treat the input as a time sequence of features. SpecAugment-style masking means we hide random parts of that sequence during training.

A simple version is time masking where we pick a random time interval and set those feature vectors to zero. If the full sequence length is T , and we mask a fraction like $0.2T$, then we hide about 20% of the time steps. we do this because it stops the model from memorizing tiny cues that only appear in one dataset or one speaker. It pushes the model to use broader context, like overall rhythm and energy shape, which are more stable across speakers.

3.11 Feature extraction: Wav2Vec2FeatureExtractor

Raw waveform arrays are not fed directly as plain Python lists. We convert them into tensors, normalize them, and pad them to a common length within a batch.

Normalization is usually something like shift the signal so it has mean 0 and scale so it has a standard scale. A basic z-normalization idea is:

$$\underbrace{\tilde{w}[i]}_{\text{normalized sample}} = \frac{\underbrace{w[i]}_{\text{original}} - \underbrace{\mu_w}_{\text{mean of waveform}}}{\underbrace{\sigma_w}_{\text{std of waveform}}}$$

which makes training more stable because the model sees inputs in a consistent range.

Padding means if one waveform is shorter, we add zeros to match the longest in the batch. We also create an attention mask so the model knows which parts are real and which parts are padding.

3.12 The model: why Wav2Vec2 is a good starting point

Why pretrained speech models help

Training a deep speech model from scratch needs huge amounts of audio. Wav2Vec2 is pretrained on large speech datasets, so it already knows a lot about speech, phonetics, transitions, and patterns over time.

Fine-tuning means we take that general speech knowledge and adapt it to our task.

What Wav2Vec2 does internally (high level, but grounded)

Wav2Vec2 starts with a convolutional feature encoder that turns raw waveform samples into a shorter sequence of feature vectors, then a transformer context network processes that sequence and builds richer representations that include context from surrounding time steps.

The output can be written as hidden states:

$$\underbrace{h}_{\text{hidden states}} \in \underbrace{R^{B \times T \times D}}_{\text{batch} \times \text{time} \times \text{feature dim}}$$

where B is batch size, T is the number of time steps after the conv encoder, and D is the feature dimension.

Each h_t is a vector that summarizes “what the model thinks is happening at time step t ”.

3.13 GAT-style fusion: why we add it and what it does

A basic classifier often takes the hidden states and averages them over time. That works, but it can miss something important for speaking style, tone is often built by a pattern across time. A single moment might not be enough.e.g. excitement might show up as repeated pitch rises, quick rhythm, and bursts of energy. Authoritative tone might show up as firm emphasis patterns and steady intensity.

So we add a mechanism that can learn which time steps matter most and how they relate.

A Graph Attention Network (GAT) is a way to compute attention weights between nodes in a graph. Here we treat each time step as a node, we can imagine a fully connected graph where every time step can attend to every other time step.

In a simple dot-product style attention, we can define a similarity score as:

$$\underbrace{e_{ij}}_{\text{similarity between time } i \text{ and } j} = \underbrace{h_i}_{\text{state at } i} \cdot \underbrace{h_j}_{\text{state at } j}$$

Then we turn these scores into weights with softmax:

$$\underbrace{\alpha_{ij}}_{\text{attention weight}} = \frac{\underbrace{\exp(e_{ij})}_{\text{positive score}}}{\underbrace{\sum_{k=1}^T \exp(e_{ik})}_{\text{normalize over all } k}}$$

The weights α_{ij} tell us how much time step i should “listen” to time step j . Then we compute a fused representation as:

$$\underbrace{\tilde{h}_i}_{\text{fused state at } i} = \underbrace{\sum_{j=1}^T \alpha_{ij} h_j}_{\text{weighted combination over time}}$$

Now we still need one vector for classification, a common approach is to pool over time. We use both mean pooling and max pooling, then concatenate them as:

$$\underbrace{h_{\text{mean}}}_{\text{mean pooled}} = \underbrace{\frac{1}{T} \sum_{i=1}^T \tilde{h}_i}_{\text{average sum fused states}} \quad \underbrace{h_{\text{max}}}_{\text{max pooled}} = \underbrace{\max_{i \in \{1, \dots, T\}} \tilde{h}_i}_{\text{take max per feature dim}}$$

Then:

$$\underbrace{h_{\text{pool}}}_{\text{final pooled vector}} = \underbrace{[h_{\text{mean}} \parallel h_{\text{max}}]}_{\text{concatenate}}$$

and finally we map to logits for three classes:

$$\underbrace{z}_{\text{logits}} = \underbrace{W}_{\text{linear weights}} \underbrace{h_{\text{pool}}}_{\text{pooled vector}} + \underbrace{b}_{\text{bias}} \quad \text{where} \quad \underbrace{z \in R^3}_{\text{three class scores}}$$

Logits are not probabilities yet. We convert them to probabilities with softmax:

$$\underbrace{\hat{y}_c}_{\text{predicted prob for class } c} = \frac{\underbrace{\exp(z_c)}_{\text{positive score}}}{\underbrace{\sum_{j=1}^3 \exp(z_j)}_{\text{normalize over 3 classes}}}$$

this helps because it gives the model a structured way to focus on the parts of speech that carry style, and to combine cues across time, which is exactly what prosody is.

3.14 LoRA fine-tuning: learning with fewer trainable parameters

Fine-tuning the full Wav2Vec2 model can be expensive and can overfit when the dataset is not massive. LoRA (Low-Rank Adaptation) is a trick to update the model without updating every parameter.

The basic idea is as follows: instead of changing a big weight matrix W directly, we keep W frozen and learn a small update. That update is written as a product of two thin matrices:

$$\underbrace{W'}_{\text{effective weight}} = \underbrace{W}_{\text{frozen base}} + \underbrace{\Delta W}_{\text{learned update}} \quad \text{with} \quad \underbrace{\Delta W}_{\text{update}} = \underbrace{B}_{\text{tall matrix}} \underbrace{A}_{\text{wide matrix}}$$

If $W \in R^{d \times k}$, then we choose a small rank r (we used $r = 16$) and set:

$$\underbrace{B}_{d \times r} \in R^{d \times r} \quad \underbrace{A}_{r \times k} \in R^{r \times k}$$

Why this saves compute: r is much smaller than d or k , so A and B together have far fewer parameters than W .

this helps generalization because it restricts the update to a simpler form so the model is less likely to memorize noise from the training set.

3.15 Loss function: why KL divergence fits mixup and soft labels

Because mixup creates soft labels, we want a loss that compares probability distributions, not just hard class IDs. KL divergence does exactly that.

For one sample with target distribution y and predicted distribution $\hat{y}$:

$$\underbrace{D_{\text{KL}}(y||\hat{y})}_{\text{KL divergence}} = \underbrace{\sum_{c=1}^3}_{\text{sum over classes}} \underbrace{y_c}_{\text{target prob}} \cdot \log \left(\frac{\underbrace{y_c}_{\text{target}}}{\underbrace{\hat{y}_c}_{\text{prediction}}} \right)$$

Now we include class weights, with soft labels, the “class weight for the sample” is not just one class. A way is to use the expected class weight under the soft label as follows:

$$\underbrace{w_{\text{sample}}}_{\text{sample weight}} = \underbrace{\sum_{c=1}^3}_{\text{sum over classes}} \underbrace{y_c}_{\text{soft label}} \cdot \underbrace{w_c}_{\text{class weight}}$$

Then the weighted sample loss is

$$\underbrace{L_{\text{sample}}}_{\text{loss for one sample}} = \underbrace{w_{\text{sample}}}_{\text{balance factor}} \cdot \underbrace{D_{\text{KL}}(y||\hat{y})}_{\text{distribution mismatch}}$$

And the batch loss is the mean over the batch as

$$\underbrace{L_{\text{batch}}}_{\text{training loss}} = \underbrace{\frac{1}{B}}_{\text{average over batch}} \underbrace{\sum_{j=1}^B L_{\text{sample},j}}_{\text{sum losses}}$$

this matters for an app because it pushes the model to produce probabilities that match soft boundaries. That is closer to reality because speaking style is not always pure, and the app should handle in-between cases gracefully.

3.16 Training setup: Trainer, hyperparameter search, scheduler, and early stopping

We train using Hugging Face’s **Trainer**. The Trainer handles the training loop batching, forward pass, loss computation, backprop, and evaluation at the end of epochs, using the Trainer reduces boilerplate and makes runs easier to reproduce.

We used mixed precision (**fp16**) to speed up training and reduce memory use. The idea is that many computations can be done with 16-bit floating point without hurting results too much.

We also used Optuna for hyperparameter search. A hyperparameter is a choice we set by hand rather than learn by gradient descent. Learning rate is the classic example. Optuna runs many small experiments, each with different settings, and keeps track of which settings produce the best validation metric.

The metric we optimize for is UAR (Unweighted Average Recall), because UAR matches our fairness goal. UAR is:

$$\underbrace{\text{UAR}}_{\text{balanced recall}} = \underbrace{\frac{1}{3}}_{\text{average over 3 classes}} \left(\underbrace{\frac{TP_{\text{calm}}}{TP_{\text{calm}} + FN_{\text{calm}}}}_{\text{recall calm}} + \underbrace{\frac{TP_{\text{excited}}}{TP_{\text{excited}} + FN_{\text{excited}}}}_{\text{recall excited}} + \underbrace{\frac{TP_{\text{authoritative}}}{TP_{\text{authoritative}} + FN_{\text{authoritative}}}}_{\text{recall authoritative}} \right)$$

We also use a learning rate scheduler, which changes the learning rate over time. A cosine scheduler starts higher and then gradually decreases in a smooth curve. The intuition is that early training benefitss from bigger steps while later training benefits from smaller steps.

Warmup means we do not jump to the full learning rate instantly, we ramp up in the first part of training because very large updates at the start can destabilize finetuning.

and finally, we use early stopping, which means if validation UAR does not improve for several checks, we stop training. This saves time and also reduces the chance of overfitting, which in practice we can set a maximum number of epochs (like 20) but

allow early stopping to end earlier if gains flatten. This matches what we want in a practical system: stop when the model is no longer learning meaningful improvements.

3.17 Turning model outputs into app feedback

When the model runs on a clip, it outputs probabilities as:

$$\underbrace{\hat{\mathbf{y}}}_{\text{predicted distribution}} = \left[\underbrace{\hat{y}_0}_{\text{calm}}, \underbrace{\hat{y}_1}_{\text{excited}}, \underbrace{\hat{y}_2}_{\text{authoritative}} \right]$$

A simple “confidence score” for the predicted class:

$$\underbrace{\text{score}}_{\text{0 to 100}} = \underbrace{100}_{\text{scale}} \cdot \underbrace{\max(\hat{y}_0, \hat{y}_1, \hat{y}_2)}_{\text{largest probability}}$$

This is not perfect calibration by itself but it is a good starting point. For an app we can also add rules like if the max probability is low, we avoid a strong statement and instead say something like “your tone sounds mixed”.

Extra prosody features: pitch variance as a coaching hint

Even though the classifier can work directly from raw audio, it is useful to compute simple prosody features for human friendly feedback. Pitch variance is one example.

To extract pitch, we can use YIN. At a basic level YIN looks for periodicity in the waveform, meaning repeating patterns that correspond to vocal fold vibrations, a rough way to say this is that it tries different candidate periods and checks which period best matches the signal repeating itself.

Once we have pitch values $f_1, \dots, f_T$ over frames, we can compute the mean as:

$$\underbrace{\mu}_{\text{mean pitch}} = \underbrace{\frac{1}{T}}_{\text{average}} \underbrace{\sum_{t=1}^T f_t}_{\text{add all pitch frames}}$$

pitch variance:

$$\underbrace{s^2}_{\text{pitch variance}} = \underbrace{\frac{1}{T-1}}_{\text{sample scaling}} \underbrace{\sum_{t=1}^T \left(\underbrace{f_t}_{\text{pitch at } t} - \underbrace{\mu}_{\text{mean}} \right)^2}_{\text{squared deviations}}$$

High variance often means more pitch movement, and so low variance means flatter intonation.

In the app, we can connect this to a simple coaching message e.g. if the model predicts calm and pitch variance is very low, the feedback might suggest adding more intonation movement if the user's goal is to sound more engaging. If the model predicts excited but pitch variance is low, it might suggest using more rise and fall patterns, not just speaking faster or louder.

The important point is that pitch variance is not used as the main classifier. It is used as an extra hint to make feedback feel specific and actionable.

3.18 Summary: why these methods fit the project goal

taking a step back the method choices all point toward the same target:

We group emotions into three speaking styles because the app needs categories that match coaching actions. We split the data with stratification because fair evaluation needs similar class mixes everywhere. We use class weights and UAR because balanced detection matters more than raw accuracy for feedback. We use augmentation and mixup because real speech is messy and tone boundaries are not sharp. We use Wav2Vec2 because it already knows speech from pretraining and we adapt it instead of starting from scratch. We add attention-based fusion because tone is built over time, not in one instant. We use LoRA because it lets us fine-tune with fewer trainable parameters. We use KL divergence because soft labels from mixup are distributions. And we connect outputs back to coaching by using both predicted probabilities and simple prosody measurements like pitch variance.

That is the our pipeline from waveform numbers, to learned representations, to style probabilities, to human friendly feedback that supports speaking practice.

4 Results

This chapter is the part where the system stops being an idea and starts being something we can measure, because till now, we have been building one pipeline with a very practical goal that was to take a short voice clip and decide which speaking style it sounds like, so the app can give feedback that actually helps someone practice. That feedback is only useful if it is consistent and fair across the three styles. If the model is great at one style but shaky on another, then the user experience becomes confusing. So the red thread through this whole chapter is simply we keep asking, “What did the model learn where does it get confused, and what does that mean for real time speaking feedback?”

Since this is speech it also helps to keep one core idea in mind the model never hears emotions directly, it can only hear signals. A voice clip is just a changing waveform. From that waveform we can extract patterns like energy (how strong the signal is) pitch (how high or low the voice sounds) and rhythm (how timing is spaced), and voice quality (roughly how sharp or breathy it sounds). Our three labels are really three buckets of these patterns. When the model fails, it is usually because two buckets share some of the same signal patterns. That is why the plots below matter, they are not just numbers, they are a map of where the signal cues separate and where they blend together.

Lets start with the dataset split because every later plot depends on it, then we look at training loss curves to see if learning is stable and then after that we look at validation accuracy and UAR to check balanced performance during training. And finally we look into test performance with the confusion matrix, ROC curves, precision-recall curves, and a pitch-variance analysis that connects the classification back to prosody feedback.

4.1 Class distribution per split: what we trained on and why balance matters

Before training the full dataset was divided into training, validation, and test sets, the training set is what the model learns from. The validation set is what we watch during training to decide whether learning is improving on unseen data. The test set is kept untouched until the very end, and it is used once to estimate how the model behaves in a realistic “new data” situation.

The split is about 80% training, 10% validation, and 10% test, that choice is a trade between training needs enough data to learn stable patterns, but we also need enough validation and test data to measure performance without the estimate being too noisy.

We also used **stratification**, which means we do not just split randomly. We split while keeping the class mix similar across train, validation, and test. The reason is basic and important. If one split had far more authoritative samples than another, then differences in performance could come from the data mix rather than from real learning, which reduces that risk.

Figure 4.1 shows the raw counts per class.

train has 2648 calm, 2654 excited, and 2955 authoritative. Validation has 331 calm, 332 excited, and 369 authoritative. Test has 331 calm, 332 excited, and 370 authoritative.

For the training total:

$$\underbrace{N_{\text{train}}}_{\text{train total}} = \underbrace{2648}_{\text{calm}} + \underbrace{2654}_{\text{excited}} + \underbrace{2955}_{\text{authoritative}} = \underbrace{\left(\underbrace{2648 + 2654}_{\substack{\text{calm} \\ \text{excited}}} \right)}_{\substack{5302 \\ \text{calm+excited}}} + \underbrace{2955}_{\text{authoritative}} = \underbrace{8257}_{\text{train total}}$$

For the validation total:

$$\underbrace{N_{\text{val}}}_{\text{validation total}} = \underbrace{331}_{\text{calm}} + \underbrace{332}_{\text{excited}} + \underbrace{369}_{\text{authoritative}} = \underbrace{\left(\underbrace{331 + 332}_{\substack{\text{calm} \\ \text{excited}}} \right)}_{\substack{663 \\ \text{calm+excited}}} + \underbrace{369}_{\text{authoritative}} = \underbrace{1032}_{\text{validation total}}$$

For the test total:

$$\underbrace{N_{\text{test}}}_{\text{test total}} = \underbrace{331}_{\text{calm}} + \underbrace{332}_{\text{excited}} + \underbrace{370}_{\text{authoritative}} = \underbrace{\left(\underbrace{331 + 332}_{\substack{\text{calm} \\ \text{excited}}} \right)}_{\substack{663 \\ \text{calm+excited}}} + \underbrace{370}_{\text{authoritative}} = \underbrace{1033}_{\text{test total}}$$

Across all splits:

$$\underbrace{N}_{\text{all samples}} = \underbrace{N_{\text{train}}}_{\substack{8257 \\ \text{train}}} + \underbrace{N_{\text{val}}}_{\substack{1032 \\ \text{val}}} + \underbrace{N_{\text{test}}}_{\substack{1033 \\ \text{test}}} = \underbrace{\left(\underbrace{8257}_{\text{train}} + \underbrace{1032}_{\text{val}} \right)}_{\substack{9289 \\ \text{train+val}}} + \underbrace{1033}_{\text{test}} = \underbrace{10322}_{\text{all samples}}$$

Now the training fraction:

$$\underbrace{\text{train fraction}}_{\text{share used for learning}} = \frac{\underbrace{N_{\text{train}}}_{\substack{8257 \\ \text{train}}}}{\underbrace{N}_{\substack{10322 \\ \text{all}}}} = \frac{\underbrace{8257}_{\text{train}}}{\underbrace{10322}_{\text{all}}} \approx \underbrace{0.80}_{\text{about 80\%}}$$

So the split is very close to 80/10/10, and because of stratification the class proportions stay almost the same across splits.

How balanced is it, really?

Even though the classes are close, authoritative is the largest class in every split. To see how big the difference is, we can compute proportions inside the training set.

For calm:

$$\underbrace{p_{\text{calm}}}_{\text{calm proportion in train}} = \frac{\underbrace{2648}_{\text{calm}}}{\underbrace{8257}_{\text{train total}}} \approx \underbrace{0.3206}_{\text{about 32.1\%}}$$

For excited:

$$\underbrace{p_{\text{excited}}}_{\text{excited proportion in train}} = \frac{\underbrace{2654}_{\text{excited}}}{\underbrace{8257}_{\text{train total}}} \approx \underbrace{0.3214}_{\text{about 32.1\%}}$$

For authoritative:

$$\underbrace{p_{\text{authoritative}}}_{\text{authoritative proportion in train}} = \frac{\underbrace{2955}_{\text{authoritative}}}{\underbrace{8257}_{\text{train total}}} \approx \underbrace{0.3579}_{\text{about 35.8\%}}$$

The imbalance is not big but it still exists authoritative is about 36% while calm and excited are about 32%, a simple way to say “how much bigger is the biggest class than the smallest class” is a ratio:

$$\underbrace{\text{size ratio}}_{\text{biggest/smallest}} = \frac{\underbrace{2955}_{\text{authoritative}}}{\underbrace{2648}_{\text{calm}}} \approx \underbrace{1.12}_{\text{about 12\% bigger}}$$

Why a mild imbalance still matters for training

Training is guided by loss which is just a number that measures how wrong the model is. The optimizer updates the model parameters to make that number smaller. The total loss is built by adding up contributions from many training examples. If one class appears more often, then mistakes on that class appear more often in the sum. That can quietly push the model toward predicting that class a bit more than it should.

This is the reason for using class weights, because class weights do not change the data. They change how strongly each example counts when the loss is added up. The goal is not to punish authoritative. The goal is to stop it from dominating the learning signal just because it has more examples:

$$\underbrace{w_i}_{\text{weight for class } i} = \frac{\underbrace{N}_{\text{total train samples}}}{\underbrace{K}_{\text{number of classes}} \cdot \underbrace{N_i}_{\text{samples in class } i}}$$

Here, $N = 8257$ and $K = 3$.

Calm weight:

$$\underbrace{w_{\text{calm}}}_{\text{calm weight}} = \frac{\overbrace{8257}^N}{\underbrace{3}_K \cdot \underbrace{2648}_{N_{\text{calm}}}} = \frac{\overbrace{8257}^N}{\underbrace{3}_K \cdot \underbrace{2648}_{N_{\text{calm}}}} = \frac{\overbrace{8257}^N}{\underbrace{7944}_{N_{\text{calm}}}} \approx \underbrace{1.04}_{\text{slightly above 1}}$$

Excited weight:

$$\underbrace{w_{\text{excited}}}_{\text{excited weight}} = \frac{\overbrace{8257}^N}{\underbrace{3}_K \cdot \underbrace{2654}_{N_{\text{excited}}}} = \frac{\overbrace{8257}^N}{\underbrace{3}_K \cdot \underbrace{2654}_{N_{\text{excited}}}} = \frac{\overbrace{8257}^N}{\underbrace{7962}_{N_{\text{excited}}}} \approx \underbrace{1.04}_{\text{slightly above 1}}$$

Authoritative weight:

$$\underbrace{w_{\text{authoritative}}}_{\text{authoritative weight}} = \frac{\overbrace{8257}^N}{\underbrace{3}_K \cdot \underbrace{2955}_{N_{\text{authoritative}}}} = \frac{\overbrace{8257}^N}{\underbrace{3}_K \cdot \underbrace{2955}_{N_{\text{authoritative}}}} = \frac{\overbrace{8257}^N}{\underbrace{8865}_{N_{\text{authoritative}}}} \approx \underbrace{0.93}_{\text{slightly below 1}}$$

So calm and excited mistakes get scaled up a bit, and authoritative mistakes get scaled down a bit, that small adjustment fits the project goal because the app should detect all speaking styles, not only the most common one.

One more note for the thesis is even if class counts are balanced, datasets like RAVDESS and CREMA-D can still differ in recording conditions, acting style, and speaker traits, and that kind of difference can create bias that is not visible in a class-count bar chart. In other words, “balanced labels” does not automatically mean “balanced difficulty”, it just means we removed one obvious source of unfairness.

4.2 Training and validation loss: what “loss” is measuring and why the curve matters

The model was fine-tuned for 5 epochs, one epoch means one full pass through the training set. We did not keep training forever because validation improvements

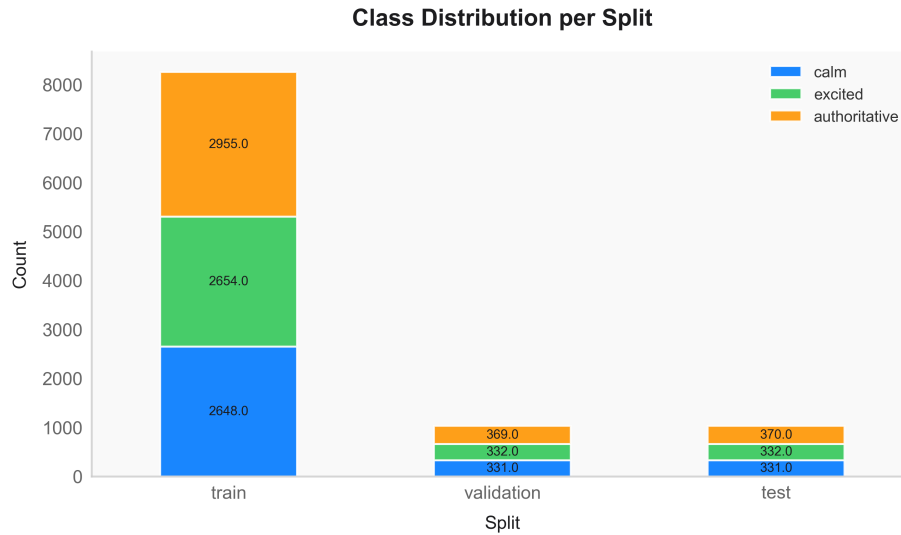


Figure 4.1: Class distribution per split. Authoritative is the largest class in every split, which motivates class weighting and balanced metrics like UAR.

became small after the early epochs. With a strong pretrained model like Wav2Vec2, it is common to see fast early gains. The base model already knows a lot about speech sounds. Fine-tuning mainly teaches it how to map those speech patterns onto our three labels.

Loss from first principles

A loss is a number that measures the mismatch between what the model predicts and what the label says it should predict, since we use mixup and soft labels, it makes sense to treat labels as probability distributions rather than hard one-hot vectors. That is why KL divergence fits naturally. KL divergence compares two probability distributions.

For one sample, the model outputs:

$$\underbrace{\hat{\mathbf{y}}}_{\text{predicted probabilities}} = \left[\underbrace{\hat{y}_0}_{\text{calm}}, \underbrace{\hat{y}_1}_{\text{excited}}, \underbrace{\hat{y}_2}_{\text{authoritative}} \right] \quad \text{with} \quad \underbrace{\hat{y}_0 + \hat{y}_1 + \hat{y}_2}_{\text{sum of probs}} = \underbrace{1}_{\text{a distribution}}$$

And the target label as:

$$\underbrace{\mathbf{y}}_{\text{target distribution}} = \begin{bmatrix} \underbrace{y_0}_{\text{calm}} , \underbrace{y_1}_{\text{excited}} , \underbrace{y_2}_{\text{authoritative}} \end{bmatrix} \quad \text{with} \quad \underbrace{y_0 + y_1 + y_2}_{\text{sum of target probs}} = \underbrace{1}_{\text{a distribution}}$$

One common form of KL divergence is:

$$\underbrace{D_{\text{KL}}(\mathbf{y} \parallel \hat{\mathbf{y}})}_{\text{how far prediction is from target}} = \sum_{c=0}^2 \underbrace{y_c}_{\text{target prob}} \cdot \log \left(\frac{\underbrace{y_c}_{\text{target}}}{\underbrace{\hat{y}_c}_{\text{prediction}}} \right)$$

Then class weighting scales the loss depending on the true class:

$$\underbrace{L_{\text{sample}}}_{\text{loss for one sample}} = \underbrace{w_i}_{\text{class weight}} \cdot \underbrace{D_{\text{KL}}(\mathbf{y} \parallel \hat{\mathbf{y}})}_{\text{KL mismatch}}$$

the loss shown per epoch is the mean over batches:

$$\underbrace{L_{\text{epoch}}}_{\text{epoch loss}} = \frac{1}{\underbrace{B}_{\text{number of batches}}} \sum_{b=1}^B \underbrace{L_b}_{\text{batch loss}}$$

That last line matters because if you accidentally log only the last batch loss instead of the mean, the curve can look jumpy.

What the loss curve says in our run

Validation loss drops from about 0.924 to about 0.770. The absolute drop is:

$$\underbrace{\Delta L_{\text{val}}}_{\text{validation loss drop}} = \underbrace{0.924}_{\text{start}} - \underbrace{0.770}_{\text{end}} = \underbrace{0.154}_{\text{drop}}$$

The relative drop is:

$$\underbrace{\text{relative drop}}_{\text{drop compared to the start}} = \frac{\underbrace{0.154}_{\text{drop}}}{\underbrace{0.924}_{\text{start}}} \approx \underbrace{0.1667}_{\text{about 16.7\%}}$$

Validation improves which means the model is learning patterns that transfer to unseen data, after about epoch 2, the improvements get smaller, which looks like an early plateau. which fits the idea that fine-tuning quickly captures the strongest cues, then runs into the harder border cases where classes overlap.

The training loss values shown are unusual because they rise and then suddenly drop, this is not the typical smooth “train down, val down” picture. The important thing is what this does and does not mean.

It does not look like classic overfitting, because validation loss does not rise, because in classic overfitting, training loss keeps getting smaller while validation loss starts getting bigger. Here, validation loss keeps decreasing.

Two simple explanations fit this, firstly, training-time augmentation can make training data objectively harder than validation data, mixup blends examples and makes labels soft. Noise and pitch shifts can distort clean cues. That can cause training loss to be higher even while learning is good, because the model is trying to solve a noisier version of the task during training.

Secondly, the training curve can be distorted by how it is logged. If one epoch logs the mean and another epoch logs the last batch, or if the augmentation pipeline is switched off for evaluation at the end of an epoch, the loss can appear to “jump”.

An honest way to write this is validation loss shows steady improvement and then flattens, while training loss is unstable, likely due to augmentation difficulty and or logging inconsistency. Because of that, the strongest evidence for learning should come from validation metrics (accuracy and UAR) and from the final test results.

4.3 Validation accuracy and UAR: tracking learning in a way that matches the app

Accuracy is probabvly the simplest, the fraction of validation samples predicted correctly:

$$\underbrace{\text{Accuracy}}_{\text{fraction correct}} = \frac{\underbrace{\text{\#correct}}_{\text{right predictions}}}{\underbrace{\text{\#total}}_{\text{all predictions}}}$$

But accuracy can hide class specific problems, that is why we also report UAR, which stands for Unweighted Average Recall. UAR starts with recall for one class.



Figure 4.2: Training and validation loss. Validation loss decreases (about 16.7%), then flattens. Training loss fluctuates strongly, which can happen when training data is made harder by augmentation and mixup.

Recall answers: “Out of all true samples of this class, how many did we correctly find?”

For class c :

$$\underbrace{\text{Recall}_c}_{\text{catch rate for class } c} = \frac{\overbrace{TP_c}^{\text{true positives}}}{\underbrace{TP_c}_{\text{true positives}} + \underbrace{FN_c}_{\text{missed positives}}}$$

Then UAR is the average recall over the three classes:

$$\underbrace{\text{UAR}}_{\text{balanced recall}} = \frac{\underbrace{\text{Recall}_{\text{calm}}}_{\text{class 0}} + \underbrace{\text{Recall}_{\text{excited}}}_{\text{class 1}} + \underbrace{\text{Recall}_{\text{authoritative}}}_{\text{class 2}}}{\underbrace{3}_{\text{three classes}}}$$

UAR is a good fit for the app because the app should not silently fail for one style. If excitement feedback is weak, the user who practices enthusiasm gets worse guidance than the user who practices calmness. UAR pushes us to measure that fairness directly.

In the validation plot the metric rises from about 0.556 at epoch 1 to about 0.677 at epoch 5. therefore total gain is:

$$\underbrace{\Delta}_{\text{total gain}} = \underbrace{0.677}_{\text{epoch 5}} - \underbrace{0.556}_{\text{epoch 1}} = \underbrace{0.121}_{\text{gain}}$$

The first jump is the biggest:

$$\underbrace{\Delta_{1 \rightarrow 2}}_{\text{epoch 1 to 2}} = \underbrace{0.646}_{\text{epoch 2}} - \underbrace{0.556}_{\text{epoch 1}} = \underbrace{0.090}_{\text{big early jump}}$$

Later jumps shrink:

$$\underbrace{\Delta_{4 \rightarrow 5}}_{\text{epoch 4 to 5}} = \underbrace{0.677}_{\text{epoch 5}} - \underbrace{0.672}_{\text{epoch 4}} = \underbrace{0.005}_{\text{tiny late gain}}$$

So again we see the same story fast learning early, then diminishing returns.

An important visual is that UAR stays close to accuracy, which is a good sign for balance. If accuracy rose while UAR stayed much lower, it might mean that the model is doing well mostly by leaning on an easier or larger class. Here, the closeness supports the idea that class weighting and the overall training setup are helping keep performance more even across the styles.

4.4 Confusion matrix: turning test errors into a clear story

The confusion matrix is one of the most useful plots for this project, because it does not just say “how good” the model is. It shows exactly how it fails. That directly tells us what kind of wrong feedback the app might give.

A confusion matrix compares true labels (rows) with predicted labels (columns). Because the matrix is normalized by rows, each row sums to 1.00. So each cell can be read as a conditional probability:

$$\underbrace{\text{cell}(i, j)}_{\text{row-normalized entry}} = \underbrace{P(\hat{y} = j \mid y = i)}_{\text{predicted } j \text{ given true } i}$$

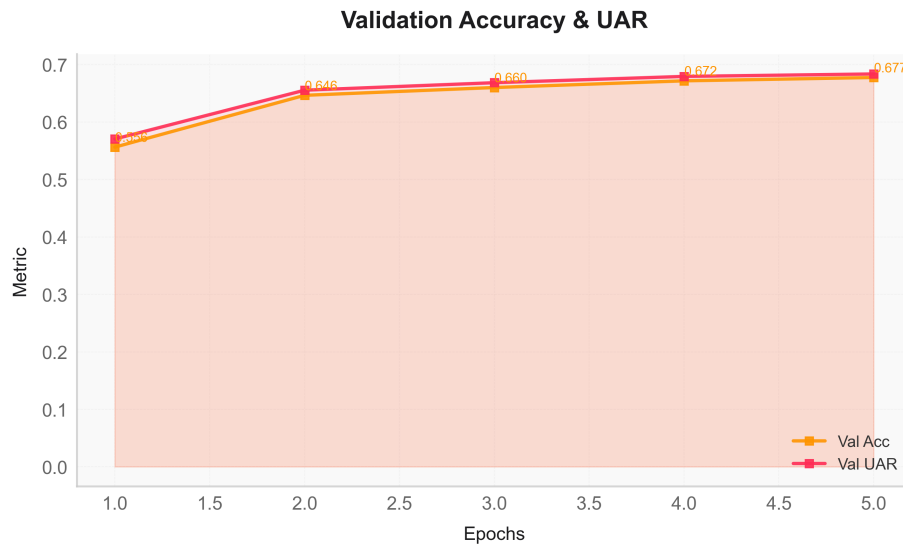


Figure 4.3: Validation accuracy and UAR. Most improvement happens early, which matches fine-tuning a pretrained speech model. UAR staying close to accuracy suggests the model is not only succeeding on one class, which matters for fair feedback in the app.

Here y is the true label and $\hat{y}$ is the predicted label and the symbol “|” means “given that”.

For true calm:

$$\underbrace{0.80}_{\hat{y}=\text{calm}} + \underbrace{0.06}_{\hat{y}=\text{excited}} + \underbrace{0.14}_{\hat{y}=\text{authoritative}} = \underbrace{1.00}_{\text{row sum}}$$

So calm is detected well,. the main calm mistake is being called authoritative.

For true excited:

$$\underbrace{0.28}_{\hat{y}=\text{calm}} + \underbrace{0.47}_{\hat{y}=\text{excited}} + \underbrace{0.25}_{\hat{y}=\text{authoritative}} = \underbrace{1.00}_{\text{row sum}}$$

Excited is the hardest. Only 47% is correctly detected, and the rest splits in two directions.

For true authoritative:

$$\underbrace{0.20}_{\hat{y}=\text{calm}} + \underbrace{0.07}_{\hat{y}=\text{excited}} + \underbrace{0.73}_{\hat{y}=\text{authoritative}} = \underbrace{1.00}_{\text{row sum}}$$

Authoritative is fairly strong, but 20% is softened into calm.

Recall and UAR from the confusion matrix

Because this matrix is row-normalized, the diagonal entries are exactly the per-class recalls:

$$\underbrace{R_{\text{calm}}}_{\text{recall for calm}} = \underbrace{0.80}_{\text{diagonal}} \quad \underbrace{R_{\text{excited}}}_{\text{recall for excited}} = \underbrace{0.47}_{\text{diagonal}} \quad \underbrace{R_{\text{authoritative}}}_{\text{recall for authoritative}} = \underbrace{0.73}_{\text{diagonal}}$$

Then UAR is:

$$\underbrace{\text{UAR}}_{\text{test}} = \frac{\underbrace{R_{\text{calm}}}_{0.80} + \underbrace{R_{\text{excited}}}_{0.47} + \underbrace{R_{\text{authoritative}}}_{0.73}}{\underbrace{3}_{\text{classes}}} = \frac{\underbrace{\left(\underbrace{0.80}_{\text{calm}} + \underbrace{0.47}_{\text{excited}} + \underbrace{0.73}_{\text{authoritative}} \right)}_{\substack{\text{sum of recalls} \\ 2.00}}}{\underbrace{3}_{\text{classes}}} = \underbrace{0.666\dots}_{\text{repeating}} \approx \underbrace{0.67}_{\text{about 67\%}}$$

The miss rate (error rate per class) is then $1 - \text{recall}$:

$$\begin{aligned} \underbrace{E_{\text{calm}}}_{\text{miss rate calm}} &= \underbrace{1}_{\text{all calm}} - \underbrace{0.80}_{\text{caught calm}} = \underbrace{0.20}_{20\%} \\ \underbrace{E_{\text{excited}}}_{\text{miss rate excited}} &= \underbrace{1}_{\text{all excited}} - \underbrace{0.47}_{\text{caught excited}} = \underbrace{0.53}_{53\%} \\ \underbrace{E_{\text{authoritative}}}_{\text{miss rate authoritative}} &= \underbrace{1}_{\text{all authoritative}} - \underbrace{0.73}_{\text{caught authoritative}} = \underbrace{0.27}_{27\%} \end{aligned}$$

So the main weakness is measurable, excited is missed about 53% of the time.

Why these confusions make sense for our three style grouping

This is where it helps to define prosody in plain language. **Prosody** means how speech sounds, not what the words are. It includes pitch movement, energy, timing, and intonation.

Our labels are built by grouping multiple emotions into a single style, that grouping is useful for an app because it reduces the number of categories the user must think

about, but it also increases within-class variety. e.g. Excited includes happy, surprised, and fearful. Those can share high arousal, meaning strong activation in the voice but they do not always sound the same. Fearful can sometimes sound tense or sharp, which can drift toward angry like, and happy can be bright and bouncy. Some speakers also express excitement softly, which can drift toward calm cues.

That helps explain why true excited splits both ways excited is sometimes confused with authoritative because both can be high energy and intense, and excited is sometimes confused with calm because not all excited examples are strongly energetic.

Authoritative being confused as calm also fits the signal story, because some authoritative samples may be controlled and steady rather than loud and sharp. If the model relies heavily on energy and pitch movement, then a controlled authoritative clip can look closer to calm.

For the app, this tells us something practical that calm feedback is relatively reliable, authoritative feedback is decent but can be missed, and excited feedback is the risky one. If the app gives hard statements like “you sounded excited” without uncertainty, this class will cause the most confusing moments. A safer design is to use confidence thresholds and allow an “uncertain” or “mixed tone” state.

Because the confusion matrix is row-normalized, we cannot compute true precision from it alone. Precision depends on column totals in raw counts, not just row fractions. So from this plot we can discuss recall, UAR, and where errors flow, but precision must come from raw counts or from the precision-recall curves.

4.5 ROC curves: what happens when we change the decision threshold

A multiclass model outputs probabilities, the usual rule is “pick the largest probability”. But for an app, we often want to control how easily the system makes a strong claim. That is where thresholds matter.

A ROC curve is built by treating one class as “positive” and the other two classes as “negative” (one-vs-rest), then sweeping a threshold from strict to loose.

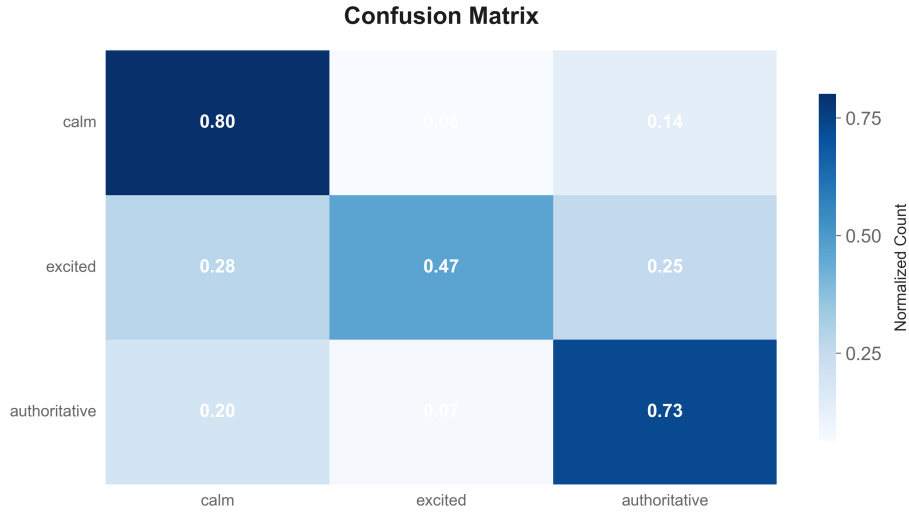


Figure 4.4: Row-normalized confusion matrix on the test set. Rows sum to 1, so each cell is $P(\hat{y} = j \mid y = i)$. Excited has the lowest recall and splits toward both calm and authoritative, matching overlap in prosody cues.

The true positive rate (TPR) is:

$$\underbrace{\text{TPR}}_{\text{catch rate}} = \frac{\underbrace{TP}_{\text{true positives}}}{\underbrace{TP}_{\text{true positives}} + \underbrace{FN}_{\text{missed positives}}}$$

The false positive rate (FPR) is:

$$\underbrace{\text{FPR}}_{\text{false alarm rate}} = \frac{\underbrace{FP}_{\text{false positives}}}{\underbrace{FP}_{\text{false positives}} + \underbrace{TN}_{\text{true negatives}}}$$

As we lower the threshold, TPR usually increases because we label more things as that class, but FPR also increases because we start labeling negatives incorrectly.

The area under the ROC curve (AUC) is:

$$\underbrace{\text{AUC}}_{\text{overall separability}} = \int_0^1 \underbrace{\text{TPR}(\text{FPR})}_{\text{TPR as FPR changes}} d(\underbrace{\text{FPR}}_{\text{x-axis}})$$

The reported AUC values are:

$$\underbrace{\text{AUC}_{\text{calm}}}_{\text{calm}} = \underbrace{0.86}_{\text{given}} \quad \underbrace{\text{AUC}_{\text{authoritative}}}_{\text{authoritative}} = \underbrace{0.85}_{\text{given}} \quad \underbrace{\text{AUC}_{\text{excited}}}_{\text{excited}} = \underbrace{0.81}_{\text{given}}$$

So overall discrimination is fine but excited is again the weakest, which matches the confusion matrix and aligns the same signal-level explanation, excited overlaps more with the other styles because it mixes multiple emotion types that can share energy with authoritative or softness with calm.

The ROC curve also hints at a practical app idea, thresholds do not need to be identical across classes. The curves show that the classes do not behave equally at low FPR, if we want to avoid false positives in an app, it makes sense to use class-specific thresholds e.g. we might keep authoritative strict to avoid wrongly accusing someone of sounding harsh, while allowing excited to be slightly looser but only when the model confidence is truly high.

If the app maps probabilities to a 0 to 100 score, then it is also worth mentioning calibration in the thesis. Calibration means that when the model says “80% confident” it should actually be correct about 80% of the time on similar data. Without calibration, scores can feel unreliable even if the classifier is decent.

4.6 Precision-recall curves: when the app should trust a prediction

Precision-recall (PR) curves are often the most directly connected to trust. Precision answers: “When the model predicts this class how often is it correct?”

Precision is:

$$\underbrace{\text{Precision}}_{\text{trustworthiness}} = \frac{\underbrace{TP}_{\text{true positives}}}{\underbrace{TP}_{\text{true positives}} + \underbrace{FP}_{\text{false positives}}}$$

Recall is:

$$\underbrace{\text{Recall}}_{\text{catch rate}} = \frac{\underbrace{TP}_{\text{true positives}}}{\underbrace{TP}_{\text{true positives}} + \underbrace{FN}_{\text{missed positives}}}$$

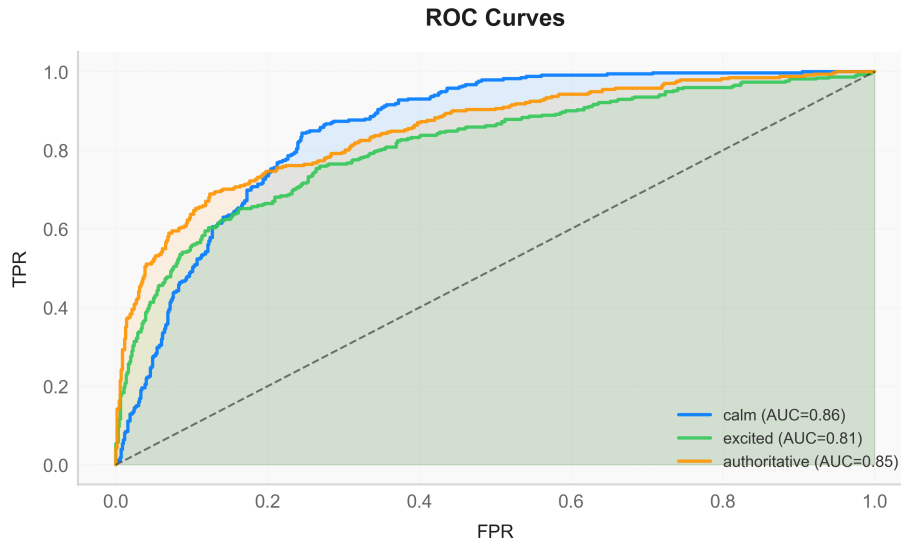


Figure 4.5: ROC curves per class. AUC values show good overall separability, with excited being the hardest class. Different curve shapes suggest that class-specific thresholds are reasonable for user-facing feedback.

A PR curve shows how precision drops as we push recall upward by lowering the threshold. Lower thresholds catch more true cases but also pull in more false positives.

Average precision (AP) summarizes the PR curve as an area as:

$$\underbrace{\text{AP}}_{\text{area under PR}} \approx \int_0^1 \underbrace{\text{Precision(Recall)}}_{\text{precision as recall changes}} d(\underbrace{\text{Recall}}_{\text{x-axis}})$$

the approximate AP values reported from the plot are:

$$\underbrace{\text{AP}_{\text{authoritative}}}_{\text{authoritative}} \approx \underbrace{0.74}_{\text{approx}} \quad \underbrace{\text{AP}_{\text{excited}}}_{\text{excited}} \approx \underbrace{0.71}_{\text{approx}} \quad \underbrace{\text{AP}_{\text{calm}}}_{\text{calm}} \approx \underbrace{0.65}_{\text{approx}}$$

A calm bucket that groups neutral, sad, and calm can be harder to pin down, because that bucket is wide, it can contain very different speaking styles that are all low arousal, and the boundary between “calm” and “not calm” becomes fuzzy. That kind of fuzziness often shows up as lower precision when we try to catch a lot of calm samples.

For the app, PR curves help choose how confident the system should be before giving a label. A false positive in coaching can be worse than a miss. If the app incorrectly says “you sounded authoritative”, that can feel unfair or discouraging. If it misses a moment of excitement, that is annoying but often less harmful, so this pushes the design toward points with higher precision, even if recall is only moderate and toward uncertainty when the model is not confident.

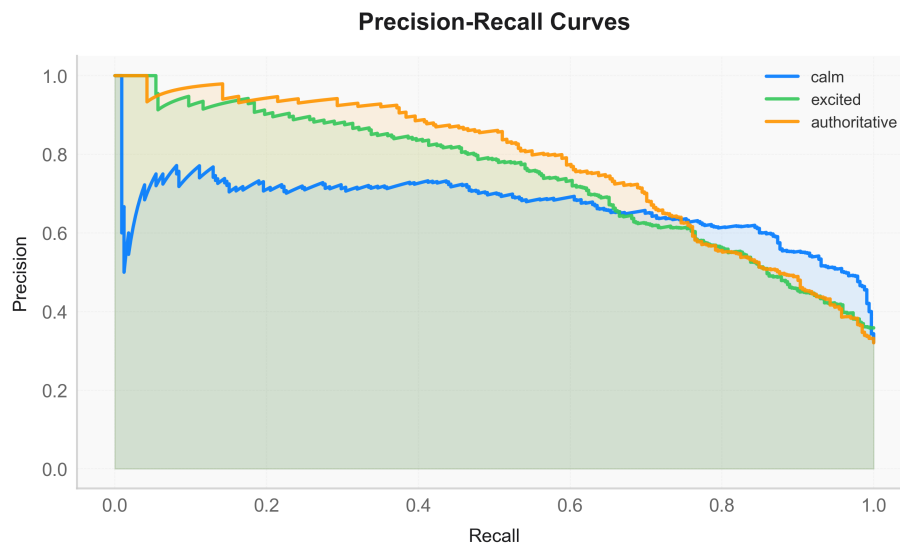


Figure 4.6: Precision-recall curves. Precision drops when we force recall to be very high, meaning that overly loose thresholds would produce many false positives in a user-facing app.

4.7 Pitch variance: connecting classification back to prosody feedback

Because the final product is a speaking coach, we also want to look at a simple prosody feature and ask whether it lines up with our style groups, so pitch is the fundamental frequency of the voice, often written as F_0 . Higher pitch means a higher sounding voice lower pitch means a deeper sounding voice. Pitch variance is a way of measuring

how much the pitch moves up and down over time. Low variance means the pitch stays more steady. High variance means the pitch moves more.

Suppose we extract a pitch value at each short time frame as:

$$\underbrace{f_1, f_2, \dots, f_T}_{\text{pitch values over } T \text{ frames}}$$

The mean pitch is:

$$\underbrace{\mu}_{\text{mean pitch}} = \underbrace{\frac{1}{T}}_{\text{average}} \underbrace{\sum_{t=1}^T f_t}_{\text{add all pitch frames}}$$

The variance is:

$$\underbrace{s^2}_{\text{pitch variance}} = \underbrace{\frac{1}{T-1}}_{\text{sample scaling}} \underbrace{\sum_{t=1}^T \left(\underbrace{f_t}_{\text{pitch at } t} - \underbrace{\mu}_{\text{mean}} \right)^2}_{\text{square deviations and add}}$$

So the boxplot compares s^2 across the three classes on the test set.

The main visible result is that the three distributions overlap heavily. The middle ranges are all in a similar band and there is no clean separation, that tells us something very grounded that pitch variance alone is not enough to separate calm, excited, and authoritative. That is not surprising if we remember what the labels contain. Authoritative speech is not always monotone. It can have strong intonation patterns that sound commanding. Calm speech can also have pitch movement if the speaker is expressive but still low-energy. Excited speech does not always mean high pitch movement either, especially if the speaker expresses excitement in a softer way.

There is also an interesting detail that is that the median pitch variance for calm appears slightly higher than excited and authoritative in this plot. That is the kind of result that is worth stating plainly instead of trying to force the expected story. A simple, honest interpretation is that our three groups are not defined by pitch movement alone, and the datasets contain acting styles that can break the usual everyday expectation.

Another important detail is the many very low outliers near zero in all classes. That often happens when pitch tracking fails or when frames are unvoiced (silence,

breath, whisper like segments). Many pitch extractors output 0 or NaN for unvoiced frames. If those frames are not handled carefully, they can distort variance.

For a thesis discussion, a clean improvement is to say that pitch features should be computed after removing unvoiced frames, and that pitch can be normalized per speaker, one common approach is log pitch:

$$\underbrace{g_t}_{\text{log pitch}} = \log(\underbrace{f_t}_{\text{pitch}})$$

Then compute the mean in log space:

$$\underbrace{\bar{g}}_{\text{mean log pitch}} = \underbrace{\frac{1}{T}}_{\text{average}} \underbrace{\sum_{t=1}^T g_t}_{\text{add all log-pitch frames}}$$

And the standard deviation in log space:

$$\underbrace{s_g}_{\text{log-pitch std}} = \sqrt{\underbrace{\frac{1}{T-1}}_{\text{sample scaling}} \underbrace{\sum_{t=1}^T \left(\underbrace{g_t}_{\log(f_t)} - \underbrace{\bar{g}}_{\text{mean log pitch}} \right)^2}_{\text{squared deviations in log space}}}$$

this helps make the feedback fairer across speakers, because different people have different natural pitch ranges. The app does not want to reward someone just because they naturally speak at a higher pitch, or punish someone just because they naturally speak at a lower pitch. Normalization pushes the feature to measure relative movement instead of absolute Hz scale.

The main takeaway is that pitch variance is a good coaching cue, but it is only one cue. The overlap in the plot supports the choice of using a deep speech model that can learn many cues at once, while still allowing us to extract pitch hints for human friendly feedback like “your intonation was more flat” or “your intonation moved more”.

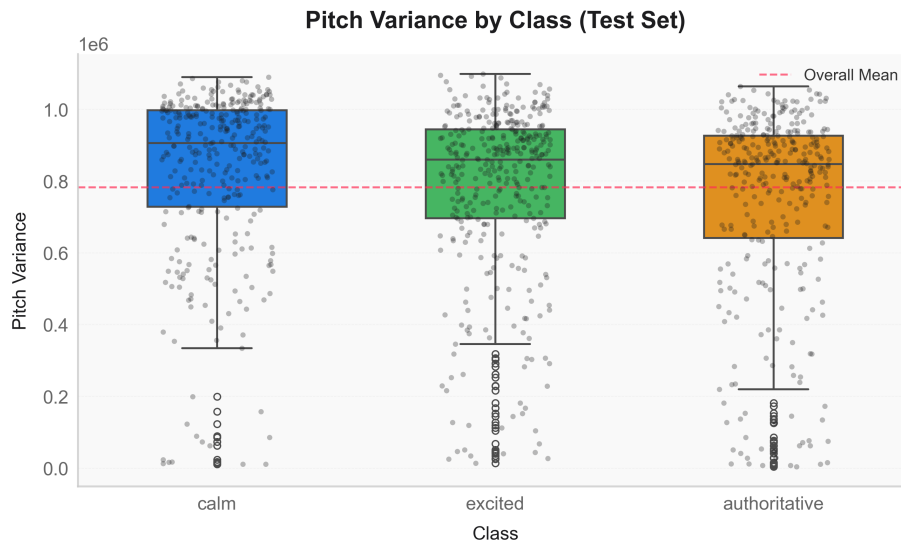


Figure 4.7: Pitch variance by class on the test set. Heavy overlap suggests pitch variance alone is a weak separator of the three styles, even though it remains useful as a coaching signal when normalized and cleaned.

4.8 Closing the loop: what these results mean for the app

The data split is clean and stratified so evaluation is fair, the class imbalance is mild, but because training loss is built from sums of per example errors, class weights still make sense to keep the learning signal more even. Validation loss decreases and then flattens, showing early learning followed by smaller gains, which is expected when fine-tuning a strong pretrained model. Validation accuracy and UAR rise quickly and then plateau, with UAR staying close to accuracy, which supports balance.

On the test set, calm is strongest, authoritative is fairly strong, and excited is the weak point. The confusion matrix shows that excited splits in two directions, which matches the idea that the excited bucket contains multiple prosody “subtypes” some closer to calm and some closer to authoritative. ROC and PR curves support the same and suggest that class thresholds and uncertainty feedback are a good idea for a user facing coach, and finally the pitch variance plot shows that pitch movement alone cannot separate the classes in a clean way, which supports using a model that

learns multiple cues from the waveform, while still using pitch features as one piece of the coaching story rather than the whole story.

5 Discussion

This project set out to build a practical foundation for automatic speaking style feedback from voice, instead of trying to predict many fine grained emotions, the work focuses on three broad and actionable styles that is, calm, excited, and authoritative. This choice matches the coaching goal, because these categories connect more directly to prosody patterns a user can practice, like pitch movement, energy, and rhythm.

The results show that the model learns meaningful differences between the three styles and generalizes beyond the training data. At the same time the error reveal an important limitation where *excited* is the hardest style to detect and so the confusion matrix shows that excited speech is often predicted as calm or authoritative. This makes sense if we come from a first principles thinking, because excited speech can vary a lot inside the same label group (happy, surprised, fearful), and parts of that group share acoustic cues with the other styles e.g. excited and authoritative can both have high energy, while some excited samples may be softer and drift toward calm. In other words, the boundary is naturally fuzzy because the prosody signals overlap.

The pitch variance analysis gives the same idea, pitch variance alone does not clearly separate the three styles, which suggests that single handcrafted cues are not enough by themselves, this increases the motivation for using a pretrained speech model that can combine many cues at once. But it also suggests that user facing feedback should be confidence-aware. If the model is uncertain, it is safer to present feedback as “mixed” rather than forcing a hard label, especially for excited.

The limitations that remain are that: training data is acted and recorded under controlled conditions, so performance may drop for real user audio with different microphones, accents, and environments. The label grouping also increases within class variation, which can reduce separability, especially for excited. And the full application interface was not completed within the project time frame, so the thesis

should be viewed as a working prototype of the core model and pipeline rather than a finished product.

Future work can improve both accuracy and usefulness. On the modeling side, excited could be made more internally consistent by refining the mapping (for example, separating fearful from other excited emotions during analysis) or by adding training emphasis on the most confused pair (excited vs. authoritative), and we could also always add more training data which would easily improve the accuracy. On the product side, a realtime app should include class thresholds, probability calibration for score interpretation, and simple prosody explanations that connect model outputs to actionable practice goals.

6 Conclusion

This thesis developed and evaluated an AI pipeline for speech style recognition aimed at supporting speaking practice through feedback on voice modulation and tone. Using RAVDESS and CREMA-D, emotions were grouped into three coaching-oriented styles (calm, excited, authoritative), and a pretrained Wav2Vec2-based model was fine-tuned with augmentations and class balancing. The evaluation shows that the system can distinguish the three styles with reasonable performance (Can easily be improved as metnioned), with the strongest results for calm and authoritative and the largest weakness for excited due to overlap in prosodic cues. The main contribution is therefore a tested foundation that can be integrated into a future real-time feedback interface, together with a clear understanding of where the model works well and where it needs improvement.

Bibliography

- [1] Steven R Livingstone and Frank A Russo. „The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS): A dynamic, multimodal set of facial and vocal expressions in North American English“. In: *PloS one* 13.5 (2018), e0196391.
- [2] Houwei Cao et al. „CREMA-D: Crowd-sourced emotional multimodal actors dataset“. In: *Journal of emotions* 8.4 (2014), p. 26.
- [3] Alexei Baevski et al. „wav2vec 2.0: A framework for self-supervised learning of speech representations“. In: *Advances in neural information processing systems* 33 (2020), pp. 12449–12460.